**From CH3-1.TXT**

*** **SQL> REM 300+ FUNCTIONS****

*** **SQL> REM 45 APPROX.****

*** **SQL> REM LEVELS OF FUNCTIONS : 1 SINGLE ROW 2 MULTI ROW****

*** **SQL> REM SINGLE ROW / SCALAR :****

*** **SQL> REM TYPES OF SINGLE ROW / SCALAR : 1 CHARACTER 2 NUMBER 3 DATE 4 CONVERSION 5 GENERAL****

*** **SQL> REM CHARACTER FUNCTIONS :****

*** **SQL> REM TYPES OF CHARACTER FUNCTIONS : 1 CASE MANIPULATION 2 CHARACTER MANIPULATION****

*** **SQL> REM MANIPULATION : ANY TYPE OF CHANGES****

*** **SQL> REM 1 SHRI GANESH 2 shri ganesh 3 Shri Ganesh 4 SHri GaNEeH****

*** **SQL> REM 2 CAPITAL/UPPER 2 SMALL/LOWER 3 INITCAP/CAMEL/PROPER 4 MIxed****

*** **SQL> REM LIST OF CASE MANIPULATION FUNCTIONS : 1 UPPER 2 LOWER 3 INITCAP****

*** **SQL> REM UPPER : TO CONVERT ALL ALPHABETS TO CAPITAL****

*** **SQL> REM LOWER : TO CONVERT ALL ALPHABETS TO SMALL/LOWER****

*** **SQL> REM INITCAP : TO CONVERT FIRST ALPHABET OF EACH WORD INTO CAPITAL AND REMAINING ALPHABETS TO SMALL. (IT WILL USE SPECIAL CHARACTER AS WORD SEPARATOR)****

*** **SQL> REM TYPES OF CHARACTERS : 1 ALPHABETS (A-Z,a-z) 2 DIGITS/NUMBER : 0-9 3 SPECIAL CHARACTER : ALL REMAINING****

*** **SQL> REM NESTED FUNCTIONS : FUNCTION WITHIN FUNCTION****

*** **SQL> REM RULES FOR NESTED FUNCTIONS :****

*** **SQL> REM CHARACTER MANIPULATION FUNCTIONS :****

*** **SQL> REM LIST OF CHARACTER MANIPULATION FUNCTIONS : CONCAT LPAD RPAD LTRIM RTRIM TRIM LENGTH TRANSLATE REPLACE SUBSTR INSTR****

*** **SQL> REM CONCAT : IT TAKES EXACTLY TWO ARGUMENTS.****

*** **SQL> REM IT IS USED TO CLUB/COMBINE/JOIN TWO INPUTS/ARGUMENTS/PARAMETERS/VALUES****

*** **SQL> REM CONCAT , || HAS IDENTICAL/SAME FUNCTIONALITY.****

*** **SQL> REM KING ad_pres (CONCAT)****

**From CH3-2.TXT**

*** **SQL> REM CHARACTER MANIPULATION FUNCTIONS : CONCAT LPAD RPAD LTRIM RTRIM TRIM LENGTH TRANSLATE REPLACE SUBSTR INSTR****

*** **SQL> REM LPAD : TO ADD SOME CHARACTERS OR CHARACTER COMBINATIONS TO THE LEFT OF INPUT.****

*** **SQL> REM IT TAKES 2 OR 3 ARGUMENTS.****

*** **SQL> REM DEFAULT VALUE OF 3rD ARGUMENT IS SPACE.****

*** **SQL> REM RPAD : TO ADD SOME CHARACTERS OR CHARACTER COMBINATIONS TO THE RIGHT OF INPUT.****

*** **SQL> REM IT TAKES 2 OR 3 ARGUMENTS.****

*** **SQL> REM RTRIM : IT IS USED TO REMOVE SOME CHARACTERS OR CHARACTER COMBINATIONS FROM RIGHT OF INPUT.****

*** **SQL> REM LTRIM : IT IS USED TO REMOVE SOME CHARACTERS OR CHARACTER COMBINATIONS FROM LEFT OF INPUT.****

*** **SQL> REM 1 17-JUL****

*** **SQL> REM TRIM : TO REMOVE SOME CHARACTERS FROM LEFT , RIGHT OR BOTH SIDES.****

*** **SQL> REM BY DEFAULT IT REMOVES CHARACTERS FROM BOTH SIDES.****

*** **SQL> REM IT ALLOWS US SINGLE CHARACTER TO BE REMOVED.****

*** **SQL> REM LENGTH : IT COUNTS NUMBER OF CHARACTERS IN INPUT.****

*** **SQL> REM DISPLAY THOSE WHO GET SALARY IN 5 FIGURES.****

*** **SQL> REM DATE DEFAULT FORMAT = DD-MON-RR****

*** **SQL> REM 1 DISPLAY THOSE WHOSE FIRST_NAME HAS 6 CHARACTERS****

*** **SQL> REM TRANSLATE : IT IS USED TO REPLACE OR REMOVE CHARACTER BY CHARACTER****

*** **SQL> REM TRANSLATE ACCEPTS EXACTLY 3 ARGUMENTS.****

*** **SQL> REM 1 1787****

*** **SQL> REM 2 JUL****

*** **SQL> REM 3 DISPLAY FIRST_NAME LIKE THIS : stEvEn nEEnA brUcE****

*** **SQL> REM 4 DISPLAY LAST_NAME LIKE THIS : KiNG KoCHHaR****

*** **SQL> REM 5 DISPLAY FULL_NAME LIKE THIS : STeVeN kIng NeeNa kOchhAr****

**From CH3-3.TXT**

*** **SQL> REM CHARACTER FUNCTIONS :****

*** **SQL> REM LIST OF CHARACTER FUNCTIONS : CONCAT LPAD RPAD LTRIM RTRIM TRIM LENGTH TRANSLATE REPLACE SUBSTR INSTR****

*** **SQL> REM REPLACE : IT IS USED TO REPLACE/REMOVE WORD BY WORD****

*** **SQL> REM JAN -> january , FEB -> 2 , MAR -> iii ,****

*** **SQL> REM SUBSTR : IT IS USED TO SEPARATE SOME CHARACTERS FROM LEFT , MIDDLE OR RIGHT.****

*** **SQL> REM 1 17****

*** **SQL> REM jul jUl****

*** **SQL> REM INSTR : TO FIND POSITION OF CHARACTER/STRING WITHIN WORD/SENTENCE.****

*** **SQL> REM DISPLAY THOSE FIRST_NAME WHICH CONTAINS e****

*** **SQL> REM UPPER LOWER INITCAP CONCAT LPAD RPAD LTRIM RTRIM TRIM LENGTH TRANSLATE REPLACE SUBSTR INSTR****

*** **SQL> REM NUMBER FUNCTIONS :****

*** **SQL> REM LIST OF NUMBER FUNCTIONS : POWER SQRT MOD FLOOR CEIL TRUNC ROUND****

*** **SQL> REM POWER : IT IS USED TO FIND EITHER RAISED TO OR ROOT VALUE. IF SECOND ARGUMENT IS 1 OR MORE, IT WILL RETURN RAISED TO VALUE, OTHERWISE WILL RETURN ROOT VALUE.****

*** **SQL> REM SQRT : IT IS USED TO FIND SQUARE ROOT OF ANY POSITIVE NUMBER.****

*** **SQL> REM MOD : TO FIND REMAINDER.****

*** **SQL> REM FLOOR , CEIL :****

*** **SQL> REM FLOOR : IT RETURNS EITHER SAME INTEGER OR PREVIOUS LOWER INTEGER.****

*** **SQL> REM CEIL : IT RETURNS EITHER SAME INTEGER OR NEXT HIGHER INTEGER.****

*** **SQL> REM TRUNC : IT IS USED REMOVE SOME DIGITS AFTER DECIMAL POINT AND ADDITIONALLY CONVERT SOME DIGITS TO ZERO, WHICH ARE PLACED BEFORE DECIMAL POINT.****

*** **SQL> REM ROUND :****

*** **SQL> REM NUMBER SYSTEM : DECIMAL****

*** **SQL> REM BASE : 10****

*** **SQL> REM AVAILABLE DIGITS : 0 1 2 3 4 5 6 7 8 9****

*** **SQL> REM FIRST HALF : 0 1 2 3 4****

*** **SQL> REM SECOND HALF : 5 6 7 8 9****

*** **SQL> REM IF DIGIT TO BE COMPARED, FALLS IN FIRST HALF, NO CHANGE IN PREVIOUS DIGIT****

*** **SQL> REM IF DIGIT TO BE COMPARED, FALLS IN SECOND HALF, ADD 1 TO PREVIOUS DIGIT****

**From CH3-4.TXT**

* **SQL> REM 23. DISPLAY FIRST_NAME , SALARY , MANAGER_ID , JOB_ID. REPORTS TO MANAGER_ID 100 OR 101 (USE MULTI ROW OPERATOR) AND WORKS AS EITHER MK_MAN, SA_MAN, AD_ASST (DON'T USE SPECIAL OPERATOR AS WELL AS MULTI ROW OPERATOR). SORT ROWS ON SALARY DESC.**

* **SQL> REM SINGLE ROW OPERATORS : = < > <= <> != ^=**

* **SQL> REM MULTI ROW OPERATORS : ANY/SOME ALL**

* **SQL> REM SPECIAL OPERATORS : IN BETWEEN LIKE IS NULL**

* **SQL> REM LOGICAL OPERATORS : NOT AND OR**

* **SQL> REM 1. DISPLAY DEPARTMENT_ID , MANAGER_ID , COMMISSION_PCT , FIRST_NAME. SORT RESULT ON ALL 4 COLUMNS AS SPECIFIED IN SINGLE SELECT STATEMENT. SORT ON DEPARTMENT_ID COLUMN IN ASCENDING ORDER KEEPING NULLS AT THE TOP, MANAGER_ID IN DESCENDING ORDER KEEPING NULLS AT THE BOTTOM, ASCENDING ORDER OF COMMISSION_PCT USING COLUMN POSITION AND ON FIRST_NAME COLUMN IN DESCENDING USING COLUMN ALIAS.**

* **SQL> REM 17. DISPLAY FIRST_NAME , SALARY , HIRE_DATE. EMPLOYEE SHOULD BE HIRED IN ANY YEAR OF 99,98 OR 87. DISPLAY ONLY THOSE WHO GETS 3rd LOWEST SALARY UPTO 6th LOWEST SALARY.**

* **SQL> REM 25 DISPLAY LAST_NAME , MANAGER_ID WHO REPORTS TO SOMEBODY.**

* **SQL> REM 1 DISPLAY NAME WHICH CONTAINS T%P**

* **SQL> REM _ : SINGLE CHARACTER**

* **SQL> REM % : ZERO OR MORE CHARACTERS**

* **SQL> REM DATE FUNCTIONS + DATE ARITHMETICS :**

* **SQL> REM LIST OF DATE FUNCTIONS : SYSDATE CURRENT_DATE ADD_MONTHS MONTHS_BETWEEN LAST_DAY NEXT_DAY TRUNC ROUND**

* **SQL> REM SYSDATE : IT RETURNS SERVER MACHINE'S DATE AND TIME INFO.**

* **SQL> REM CURRENT_DATE : IT RETURNS CURRENT/LOCAL/CLIENT MACHINE'S DATE AND TIME INFO.**

* **SQL> REM DATE ARITHMETIC :**

* **SQL> REM DATE HAS 7 PARTS/COMPONENTS/INTERVALS : CENTURY YEAR MONTH DATE/DAY NUMBER HOUR(24) MINUTE SECOND**

*** **SQL> REM DATE VALUES : 20 26 02 22 11 43 11****

*** **SQL> REM WE CAN ADD/SUBTRACT ALL ABOVE 7 INTERVALS TO/FROM DATE VALUE.****

*** **SQL> REM DATE ARITHMETICS : 1 DATE + INTERVAL 2 DATE - INTERVAL 3 DATE - DATE****

*** **SQL> REM 1 DAY = 24 HOURS = 24 * 60 = 1440 MINUTES = 24 * 60 * 60 = 86400 SECONDS****

*** **SQL> REM 1 DAY = 24 HOURS = 1440 MINUTES = 86400 SECONDS****

*** **SQL> REM DISPLAY SYSDATE AND ADD 2 HOURS AND SUBTRACT 10 MINUTES****

*** **SQL> REM 1 ADD 10 MINUTES TO CURRENT_DATE****

*** **SQL> REM ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MONTH-RRRR HH:MI.SS AM DAY';****

*** **SQL> REM 1 REMOVE ALL DIGITS AFTER . FROM 3456.89****

**From CH3-5.TXT**

*** **SQL> REM CENTURY TENURE YEAR QUARTER MONTH WEEK DAY HOUR MINUTE SECOND****

*** **SQL> REM 1 WEEK = 7 DAYS****

*** **SQL> REM ADD 3 WEEKS TO SYSDATE****

*** **SQL> REM 1 MONTH = ?? DAYS (28/29/30/31)****

*** **SQL> REM ADD_MONTHS : TO ADD/SUBTRACT MONTHS/QUARTERS/YEARS/TENURES/CENTURIES TO/FROM DATE VALUE****

*** **SQL> REM ADD 37 WEEKS TO SYSDATE****

*** **SQL> REM 1 QUARTER = 3 MONTHS****

*** **SQL> REM 1 YEAR = 12 MONTHS****

*** **SQL> REM 1 DECADE = 10 YEARS/120 MONTHS****

*** **SQL> REM 1 CENTURY = 100 YEARS/1200 MONTHS****

*** **SQL> REM ADD 2 MONTHS TO SYSDATE AND SUBTRACT 3 MONTHS FROM SYSDATE****

*** **SQL> REM ADD 1 QUARTER TO CURRENT_DATE****

*** **SQL> REM SUBTRACT 5 QUARTER TO CURRENT_DATE****

*** **SQL> REM ADD 2 YEARS TO SYSDATE****

*** **SQL> REM SUBTRACT 3 DECADES FROM CURRENT_DATE****

*** **SQL> REM ADD 2 CENTURIES TO SYSDATE****

**From CH3-6.TXT**

*** **SQL> REM DATE FUNCTIONS : SYSDATE CURRENT_DATE ADD_MONTHS MONTHS_BETWEEN LAST_DAY NEXT_DAY TRUNC ROUND****

*** **SQL> REM MONTHS_BETWEEN : TO FIND NUMBER OF MONTHS BETWEEN TWO DATE.****

*** **SQL> REM 1 WEEK = 7 DAYS****

*** **SQL> REM DISPLAY NUMBER OF DAYS , WEEKS , MONTHS , QUARTERS , YEARS WORKED (INTEGER ONLY)****

*** **SQL> REM 1 DISPLAY ONLY THOSE WHO HAVE COMPLETED MORE THAN 30 YEARS IN COMPANY.****

*** **SQL> REM 2 DISPLAY ALL THOSE WHOSE NUMBER/COUNT OF QUARTERS WORKED IS BETWEEN 111 AND 130.****

*** **SQL> REM LAST_DAY : IT WILL RETURN LAST DATE OF INPUT MONTH.****

*** **SQL> REM DISPLAY NUMBER OF DAYS WORKED IN JOINING MONTH.****

*** **SQL> REM 5 FIND NUMBER OF DAYS WORKED BY EACH EMPLOYEE IN JOINING MONTH.****

*** **SQL> REM NEXT_DAY : IT WILL RETURN DATE ON WHICH SPECIFIED DAY FALLS.****

*** **SQL> REM 1 COMPANY PAYS WEEKY SALARY ON EVERY SATURDAY. FIND 4th WEEKLY SALARY DATE AFTER JOINING THE COMPANY.****

*** **SQL> REM 2 COMPANY PAYS WEEKLY SALARY ON EVERY FRIDAY. FIND FIRST WEEKLY SALARY DATE.****

*** **SQL> REM 3 COMPANY PAYS WEEKLY SALARY ON EVERY MONDAY. FIND NEXT WEEKLY SALARY DATE.****

*** **SQL> REM 4 COMPANY PAYS WEEKLY SALARY ON EVERY WEDNESDAY. FIND LAST WEEKLY SALARY DATE.****

*** **SQL> REM 5 COMPANY PAYS WEEKLY SALARY ON EVERY TUESDAY. FIND 6tH WEEKLY SALARY DATE.****

**From CH3-7.TXT**

*** **SQL> REM COMPANY PAYS IN NEXT MONTH ON 3rd WEDNESDAY, FIND FIRST MONTHLY SALARY DATE.****

*** **SQL> REM COMPANY PAYS IN NEXT ON FRIDAY AFTER 10th DATE. FIND FIRST MONTHLY SALARY.****

*** **SQL> REM COMPANY PAYS IN NEXT MONTH ON FRIDAY AFTER 10th DATE. FIND FIRST MONTHLY SALARY.****

*** **SQL> REM TRUNC : WE CAN TRUNC DATE MAINLY WITH 3 INTERVALS : 1 MONTH 2 Q/QUARTER 3 YEAR****

*** **SQL> REM TRUNC DATE WITH MONTH : IT WILL RETURN SAME/INPUT MONTH'S FIRST DATE : 12-FEB -> 01-FEB , 28-JUN - 01-JUN****

*** **SQL> REM Q1 : 1JAN TO 31MAR****

*** **SQL> REM Q2 : 1APR TO 30JUN****

*** **SQL> REM Q3 : 1JUL TO 30SEP****

*** **SQL> REM Q4 : 1OCT TO 31DEC****

*** **SQL> REM TRUNC DATE WITH Q/QUARTER : IT WILL RETURN SAME/INPUT QUARTER'S FIRST DATE : 12-FEB -> 01-JAN , 28-JUN - 01-APR , 31-DEC -> 01-OCT , 16-SEP -> 01-JUL****

*** **SQL> REM TRUNC DATE WITH YEAR : IT WILL RETURN SAME/INPUT YEAR'S FIRST DATE : 12-FEB -> 01-JAN , 28-JUN - 01-JAN , 31-DEC -> 01-JAN , 16-SEP -> 01-JAN****

*** **SQL> REM 14-FEB-26 28-FEB-26****

*** **SQL> REM LAST_DAY****

*** **SQL> REM ADD_MONTHS(TRUNC('14-FEB-26','MONTH'),1) -> 1-MAR-26****

*** **SQL> REM 01-FEB 14-FEB 28-FEB****

*** **SQL> REM TRUNC(DDD,MONTH) LAST_DAY OR ADD_MONTHS(TRUNC(DDD))****

*** **SQL> REM 1 FIND NUMBER OF DAYS WORKED IN JOINING MONTH****

*** **SQL> REM 2 FIND DAY NUMBER OF MONTH ON WHICH COMPANY HIRED THE EMPLOYEE****

*** **SQL> REM 1 FIND NUMBER OF DAYS WORKED IN JOINING QUARTER****

*** **SQL> REM 2 FIND DAY NUMBER OF QUARTER ON WHICH COMPANY HIRED THE EMPLOYEE****

*** **SQL> REM 1 FIND NUMBER OF YEAR WORKED IN JOINING QUARTER****

*** **SQL> REM 1 FIND NUMBER OF DAYS WORKED IN JOINING YEAR****

*** **SQL> REM 2 FIND DAY NUMBER OF YEAR ON WHICH COMPANY HIRED THE EMPLOYEE****

*** **SQL> REM ROUND : WE CAN ROUND DATE MAINLY WITH 3 INTERVALS : 1 MONTH 2 Q/QUARTER 3 YEAR****

*** **SQL> REM MONTH : FIRST HALF : 1 TO 15 , SECOND HALF : 16 - .. (28 TO 31)****

*** **SQL> REM WHEN WE ROUND DATE WITH MONTH : IF INPUT DATE FALLS IN FIRST HALF, IT WILL RETURN SAME/INPUT MONTH'S FIRST DATE, OTHERWISE NEXT MONTH'S FIRST DATE.****

*** **SQL> REM EX. : 14-FEB -> 01-FEB , 16-FEB -> 01-MAR****

*** **SQL> REM 16-MAY-25 -> 01-JUN-25 , 16-DEC-25 -> 01-JAN-26****

*** **SQL> REM Q1 : 1JAN TO 31MAR : FIRST HALF : 01JAN TO 15FEB , SECOND HALF : 16FEB TO 31MAR****

*** **SQL> REM Q2 : 1APR TO 30JUN : FIRST HALF : 01APR TO 15MAY , SECOND HALF : 16MAY TO 30JUN****

*** **SQL> REM Q3 : 1JUL TO 30SEP : FIRST HALF : 01JUL TO 15AUG , SECOND HALF : 16AUG TO 30SEP****

*** **SQL> REM Q4 : 1OCT TO 31DEC : FIRST HALF : 01OCT TO 15NOV , SECOND HALF : 16NOV TO 31DEC****

*** **SQL> REM WHEN WE ROUND DATE WITH QUARTER : IF INPUT DATE FALLS IN FIRST HALF, IT WILL RETURN SAME/INPUT QUARTER'S FIRST DATE, OTHERWISE NEXT QUARTER'S FIRST DATE.****

*** **SQL> REM 14-FEB -> 01-JAN 16-NOV-25 -> 01-JAN-26****

*** **SQL> REM 1 YEAR = 12 MONTHS****

*** **SQL> REM FIRST HALF : 1JAN TO 30JUN , SECOND HALF : 1JUL TO 31DEC****

*** **SQL> REM WHEN WE ROUND DATE WITH YEAR : IF INPUT DATE FALLS IN FIRST HALF, IT WILL RETURN SAME/INPUT YEAR'S FIRST DATE, OTHERWISE NEXT YEAR'S FIRST DATE.****

*** **SQL> REM 1 DISPLAY THOSE WHO WERE HIRED IN FIRST HALF OF ANY MONTH. : 1 ROUND 2 TRUNC + ROUND****

*** **SQL> REM 2 DISPLAY THOSE WHO WERE HIRED IN SECOND HALF OF ANY MONTH. : 1 ROUND 2 TRUNC + ROUND 3 TRUNC + ROUND + ADD_MONTHS 4 TRUNC + ROUND + MONTHS_BETWEEN****

*** **SQL> REM 1 DISPLAY THOSE WHO WERE HIRED IN FIRST HALF OF ANY QUARTER. : 1 ROUND 2 TRUNC + ROUND****

*** **SQL> REM 2 DISPLAY THOSE WHO WERE HIRED IN SECOND HALF OF ANY QUARTER. : 1 ROUND 2 TRUNC + ROUND 3 TRUNC + ROUND + ADD_MONTHS 4 TRUNC + ROUND + MONTHS_BETWEEN****

*** **SQL> REM 3 DISPLAY THOSE WHO WERE HIRED IN FIRST HALF OF ANY YEAR. : 1 ROUND 2 TRUNC + ROUND****

*** **SQL> REM 4 DISPLAY THOSE WHO WERE HIRED IN SECOND HALF OF ANY YEAR. : 1 ROUND 2 TRUNC + ROUND 3 TRUNC + ROUND + ADD_MONTHS 4 TRUNC + ROUND + MONTHS_BETWEEN****

I have extracted all the lines starting with SQL> REM from the Chapter 4 source files, including explanations that span multiple lines. I have also updated the CH - 4 tailored report in the Studio tab to include these details.

Here are the extracted lines from the Chapter 4 files:

From CH4-1.TXT

- **SQL> REM SINGLE ROW FUNCTIONS :**

- **SQL> REM TYPES OF SINGLE ROW FUNCTIONS : 1 CHARACTER 2 NUMBER 3 DATE 4 CONVERSION 5 GENERAL**
- **SQL> REM GENERAL FUNCTIONS :**
- **SQL> REM TYPES OF GENERAL FUNCTIONS : 1 NULL RELATED 2 CONDITIONAL**
- **SQL> REM LIST OF NULL RELATED FUNCTIONS : NVL NVL2 NULLIF COALESCE**
- **SQL> REM NVL : IT TAKES/ACCETS EXACTLY TWO ARGUMENTS.**
- **SQL> REM IT WILL CHECK FOR FIRST ARGUMENT, IF IT IS NULL, IT WILL GET REPLACED WITH SECOND ARGUMENT.**
- **SQL> REM IF FIRST ARGUMENT IS NOT NULL, IT WILL RETURN SAME (FIRST ARGUMENT)**
- **SQL> REM IF DATA TYPE OF FIRST ARGUMENT IS CHARACTER, DATA TYPE OF SECOND ARGUMENT MAY BE ANY OF THE CHARACTER , NUMBER OR DATE.**
- **SQL> REM IF DATA TYPE OF FIRST ARGUMENT IS NON-CHARACTER, DATA TYPES OF BOTH ARGUMENTS MUST MATCH.**
- **SQL> REM DISPLAY 'NA' IF COMMISSION IS NULL**
- **SQL> REM 1 COMPANY DECIDES TO PAY .14 COMMISSION TO THOSE WHO DO NOT GET COMMISSION.**
- **SQL> REM 2 DISPLAY 'ZERO' IF COMMISSION_PCT IS NULL.**
- **SQL> REM 3 DISPLAY 'BOSS' IF MANAGER_ID IS NOT DECIDED.**
- **SQL> REM NVL2 : IT IS USED TO REPLACE NULL AS WELL AS NON-NULL VALUE.**
- **SQL> REM IT TAKES EXACTLY 3 ARGUMENTS.**
- **SQL> REM IF FIRST ARGUMENT IS NULL, IT WILL GET RESPLACED WITH THIRD ARGUMENT**
- **SQL> REM IF FIRST ARGUMENT IS NON-NULL, IT WILL GET RESPLACED WITH SECOND ARGUMENT**
- **SQL> REM IF DATA TYPE OF SECOND ARGUMENT IS CHARACTER, DATA TYPE OF THIRD ARGUMENT MAY BE ANY OF THE CHARACTER , NUMBER OR DATE.**
- **SQL> REM IF DATA TYPE OF SECOND ARGUMENT IS NON-CHARACTER, DATA TYPES OF SECOND AND THIRD ARGUMENTS MUST MATCH.**
- **SQL> REM DISPLAY 'EMPLOYEE' IF HE REPORTS SOMEBODY, OTHERWISE DISPLAY 'BOSS'**
- **SQL> REM COMPANY DECLARES DOUBLE COMMISSION TO THOSE WHO GET COMMISSION AND 'ZERO' FOR OTHERS.**
- **SQL> REM 1 COMPANY DECIDES TO PAY 50% EXTRA COMMISSION_PCT TO THOSE WHO GET COMMISSION AND .12 FOR OTHERS.**
- **SQL> REM 2 COMPANY DECIDES TO PAY DOUBLE COMMISSION_PCT TO THOSE WHO GET COMMISSION AND 'NO COMM' FOR OTHERS.**
- **SQL> REM NULLIF : IT TAKES EXACTLY TWO ARGUMENTS.**
- **SQL> REM IT WILL CHECK FOR EQUALITY/SIMILARITY, IF BOTH ARE EQUAL, IT WILL RETURN NULL, OTHERWISE WILL RETURN FIRST ARGUMENT.**
- **SQL> REM DATA TYPES OF BOTH ARGUMENTS MUST BE SAME.**
- **SQL> REM 1 COMPARE SECOND LAST CHARACTER OF FIRST_NAME AND SECOND CHARACTER OF FIRST_NAME.**

- **SQL> REM 2 COMPARE SECOND DIGIT OF HIRE_DATE AND SECOND LAST DIGIT OF HIRE_DATE.**
- **SQL> REM 3 COMPARE 4th CHARACTER OF PHONE_NUMBER WITH 8th CHARACTER OF PHONE_NUMBER. IF BOTH ARE SAME, DISPLAY 'IDENTICAL' OTHERWISE DISPLAY 'DIFF.'**
- **SQL> REM COAELESCE : IT TAKES INFINITE ARGUMENTS. DATA TYPES OF ALL ARGUMENTS MUST MATCH.**
- **SQL> REM IT WILL CHECK FROM LEFT TO RIGHT. IT WILL RETURN FIRST NON-NULL VALUE FROM THE LIST OF INPUTS.**
- **SQL> REM CONDITIONAL FUNCTIONS :**
- **SQL> REM LIST OF CONDITIONAL FUNCTIONS : 1 DECODE 2 CASE I 3 CASE II**
- **SQL> REM COMPANY DECLARES +1000 INCREMENT FOR ALL.**
- **SQL> REM COMPANY DECLARES INCREMENT BASED ON DEPARTMENT : 20 +500 , 50 +1000 , 80 +1500 , REMAINING +750**
- **SQL> REM COMPANY DECLARES INCREMENT BASED ON DEPARTMENT : 20 +500 , 50 +1000 , 80 +1500 , REMAINING NO INCREMENT**
- **SQL> REM 1 COMPANY DECLARES INCREMENT/DECREMENT : MANAGER_ID : 100 -> 1000 , 124 -> 1250 , 149 -> 1500 , REMAINING 750**
- **SQL> REM 2 COMPANY DECLARES INCREMENT/DECREMENT : JOB_ID : IT_PROG 20% , SA_REP 30% , ST_CLERK 40% , REMAINING 10%**
- **SQL> REM 3 COMPANY DECLARES INCREMENT/DECREMENT : DEPARTMENT_ID , MANAGER_ID : 50 124 +1500 , 60 103 -500 , 80 149 +750 , REMAINING -250**
- **SQL> REM CASE I :**
- **SQL> REM 1 COMPANY DECLARES INCREMENT/DECREMENT BASED ON : LENGTH OF FIRST_NAME : 5 +500 , 6 +750 , 8 +1000 , REMAINING NO INCREMENT**
- **SQL> REM 2 COMPANY DECLARES INCREMENT/DECREMENT BASED ON : WEEK NUMBER OF HIRE MONTH : 1 50% , 2 40% , 3 30% , 4 20% , 5 10%**
- **SQL> REM 3 COMPANY DECLARES INCREMENT/DECREMENT BASED ON : LEFT MOST VOWEL OF FIRST_NAME : A 10% , E 15% , I 20% , O 25% , U 30%**

From CH4-2.TXT

- **SQL> REM CASE II SYNTAX : THIS ALLOWS US TO WRITE ALMOST ALL CONDITIONS WHICH 'WHERE' CLAUSE SUPPORT.**
- **SQL> REM COMPANY DECLARES INCREMENT BASED ON SALARY RANGE : 0-6000 40% , 6000.01 TO 9000 30% , 9000.01 - 13000 20% , REMAINING 10%**
- **SQL> REM DECODE (SALARY ,**
- **SQL> REM CASE SALARY WHEN SINGLE VALUE**
- **SQL> REM 1 COMPANY DECLARES SALARY INCREMENT BASED ON HIRE YEAR RANGE : UPTO 90 10% , 91 TO 94 : 20% , 95 TO 97 30% , REMAINING 40%**
- **SQL> REM 2 COMPANY DECLARES SALARY INCREMENT BASED ON JOB_ID : (IT_PROG AD_VP) -> +1000 , (SA_REP ST_CLERK MK_MAN) -> +750 , REMAINING +500**

- **SQL> REM 3 COMPANY DECLARES SALARY INCREMENT BASED ON NUMBER OF YEARS WORKED : UPTO 26 20% , 27 TO 31 30% , 31 ONWARDS 40%**
- **SQL> REM 4 COMPANY DECLARES SALARY INCREMENT BASED ON HIRE QUARTER : 1 10% , 2 20% , 3 30% , 4 40%**
- **SQL> REM 5 COMPANY DECLARES SALARY INCREMENT BASED ON COUNT OF CONSONENT IN FIRST_NAME : (2 4) -> 20% , (3 5 6) -> 30% , REMAINING NO CHANGE**
- **SQL> REM CONVERSION FUNCTIONS : TO CONVERT A VALUE FROM A DATATYPE TO ANOTHER DATATYPE**
- **SQL> REM LIST OF CONVERSION FUNCTIONS : 1 TO_NUMBER 2 TO_DATE 3 TO_CHAR**
- **SQL> REM TO : CONVERT IT TO**
- **SQL> REM TO_NUMBER : TO CONVERT CHARACTER TYPE VALUE INTO NUMBER TYPE VALUE**
- **SQL> REM TO_DATE : TO CONVERT CHARACTER TYPE VALUE INTO DATE TYPE VALUE**
- **SQL> REM TO_CHAR : TO CONVERT NUMBER OR DATE TYPE VALUE INTO CHARACTER TYPE VALUE**
- **SQL> REM PURPOSES : 1 FOR CALCULATION 2 FOR COMPARISON**
- **SQL> REM TO_NUMBER SYNTAX : TO_NUMBER('value','format')**
- **SQL> REM LIST OF VALID NUMBER FORMAT CHARACTERS : 9 0 . , D G $ L U S**
- **SQL> REM . : DECIMAL POINT**
- **SQL> REM , : GROUP/THOUSAND SEPARATOR**
- **SQL> REM G DEFAULT VALUE IS ,**
- **SQL> REM D DEFAULT VALUE IS .**
- **SQL> REM NLS_SESSION_PARAMETERS / NLS : NATIONAL LANGUAGE SUPPORT**
- **SQL> REM TO CHANGE ANY NLS PARAMETER VALUE : ALTER SESSION SET parametername = 'value';**
- **SQL> REM L = LOCAL CURRENCY = $ = NLS_CURRENCY**
- **SQL> REM U = DUAL/EURO CURRENCY = $ = NLS_DUAL_CURRENCY**
- **SQL> REM TO GET € : PRESS AND HOLD DOWN LEFT 'ALT' KEY AND ENTER 157 USING NUMERICAL KEY AND RELEASE 'ALT' KEY**

From CH4-3.TXT

- **SQL> REM 1 10000 + 'Rs.2345.67'**
- **SQL> REM 2 10000 + '2345.67€' (DON'T USE . AND , AS FORMAT CHARACTER)**
- **SQL> REM 3 10000 + '2345^67'**
- **SQL> REM 4 10000 + '12.345,67'**
- **SQL> REM 5 10000 + '€12#345~67' (WITHOUT USING ALTER SESSION COMMAND)**
- **SQL> REM € = 157 , € = 159 , *ƒ* = FLORIN : CURRENCY OF SWITZERLAND**
- **SQL> REM DISPLAY THOSE WHO GET HIGHER SALARY THAN 10000**

- **SQL> REM TO_DATE : TO CONVERT CHARACTER TYPE VALUE INTO DATE TYPE VALUE.**
- **SQL> REM PURPOSE : 1 FOR CALCULATION 2 FOR COMPARISON**
- **SQL> REM DISPLAY NUMBER OF DAYS WORKED BY EACH EMPLOYEE**
- **SQL> REM TO_DATE SYNTAX : TO_DATE('value','format')**
- **SQL> REM LIST OF VALID DATE FORMAT CHARACTERS : DD MON MONTH MM RM YY YYYY RR RRRR**
- **SQL> REM 1 FIND YOUR AGE IN NUMBER OF DAYS.**
- **SQL> REM 2 CALCULATE : 12-JUN-90 - 23-AUGust-1985**
- **SQL> REM 3 CALCULATE : 11-NOVEMBER-85 - 1990-07-24**
- **SQL> REM 4 DISPLAY NUMBER OF WEEKS WORKED BY EACH EMPLOYEE**
- **SQL> REM 5 DISPLAY DIFFERENCE IN DAYS BETWEEN YOUR MOTHER AND FATHER'S BIRTH DATE (USE DATE INCOMPATIBLE FORMATS)**

From CH4-4.TXT

- **SQL> REM TO_DATE : TO CONVERT CHARACTER TYPE VALUE IN TO DATE TYPE VALUE**
- **SQL> REM DISPLAY THOSE WHO WERE HIRED BEFORE 1996**
- **SQL> REM YY : USES CURRENT CENTURY VALUE : 96 -> 2096 , 07 -> 2007**
- **SQL> REM RR : CURRENT 50 YEARS : 2000 TO 2049 + PREVIOUS 50 YEARS : 1950 TO 1999 : RR CENTURY : 1950 TO 2049**
- **SQL> REM 1 DISPLAY THOSE WHO WERE HIRED BEFORE 31#DECEMBER1995**
- **SQL> REM 2 DISPLAY THOSE WHO WERE HIRED AFTER 1/1/98**
- **SQL> REM 3 DISPLAY THOSE WHO WERE HIRE BETWEEN 1#JAN^1994 AND 31-12-98**
- **SQL> REM 4 DISPLAY ALL EXCEPT THOSE WHO WERE HIRED BETWEEN SECOND QUARTER OF 95 TO THIRD QUARTER OF YEAR 99 (USE COMPATIBLE FORMATS)**
- **SQL> REM 5 DISPLAY ALL EXCEPT THOSE WHO WERE HIRED BETWEEN SECOND HALF OF 93 TO FIRST HALF OF YEAR 98 (USE INCOMPATIBLE FORMATS)**
- **SQL> REM TO_CHAR : 1 NUMBER TO CHARACTER CONVERSION 2 DATE TO CHARACTER CONVERSION**
- **SQL> REM PURPOSE : TO DISPLAY NUMERICAL VALUE IN USER DEFINED/NON-DEFAULT FORMAT**
- **SQL> REM TO_CHAR SYNTAX : TO_CHAR('value','format')**
- **SQL> REM LIST OF VALID NUMBER FORMAT CHARACTERS : 9 0 . , D G $ L U S MI PR EEEE or eeee V**
- **SQL> REM EEEE : FOR SCIENTIFIC NOTATION / EXPONENT FORMAT**
- **SQL> REM 2.40E+04 = 2.4 * POWER(10,4)**
- **SQL> REM 1 DISPLAY SALARY LIKE THIS : 24,000.00**
- **SQL> REM 2 DISPLAY SALARY LIKE THIS : 24#000^00**
- **SQL> REM 3 DISPLAY SALARY LIKE THIS : €24.000,00**

- **SQL> REM 4 DISPLAY SALARY LIKE THIS : 24|000~00Rs. (USING 3 PARAMETER SYNTAX)**
- **SQL> REM 5 DISPLAY SALARY LIKE THIS : 2.40E+04**
- **SQL> REM 6 WRITE COMMAND WHICH WILL ASK FOR A NUMBER, IF INPUT IS POSITIVE DISPLAY LIKE THIS +24^000, IF NEGATIVE DISPLAY LIKE THIS [24#000]**

From CH4-5.TXT

- **SQL> REM TO_CHAR : TO DISPLAY DATE VALUES IN USER DEFINED FORMAT/NON-DEFAULT FORMAT.**
- **SQL> REM TO_CHAR SYNTAX : TO_CHAR('date value','date formats')**
- **SQL> REM SUN = 1, MON = 2 ... SAT = 7**
- **SQL> REM WEDNESDAY = 9 CHARACTER : RPAD**
- **SQL> REM J = JULIAN DAY NUMBER STARTING FROM 01-JAN-4712BC**
- **SQL> REM 1 15-03-2025**
- **SQL> REM 2 15-March-25**
- **SQL> REM 3 15-III-2025**
- **SQL> REM 4 Today is 7Th day of week, 15th day of month and 74th day of year.**
- **SQL> REM 5 We are in quarter 1 of year Twenty Twenty-Five.**
- **SQL> REM 6 15th March Two Thousand Twenty-Five.**
- **SQL> REM 7 Currenly we are in week 3 of month March and week 11 of year 2025.**
- **SQL> REM 8 Current time is 10:50:00pm**
- **SQL> REM 9 Current second of the day is 82123.**
- **SQL> REM 10 DISPLAY CURRENT SECOND OF THE DAY IN WORDS.**
- **SQL> REM & : IT WILL ASK EVERY TIME AND USE IT AND DISCARD IT.**
- **SQL> REM && : IT WILL ASK FIRST TIME AND WILL KEEP IT UNTIL END OF SESSION.**
- **SQL> REM SET VERI OFF : TO HIDE THE OLD/NEW MESSAGES FOR SUBSTITUTION VARIABLES.**

I have also updated the CH - 4 report in your Studio tab with these detailed notes.